Pakistan Navy Complaint Management System (CMS)

Document 04: Developer & Technical Setup Manual

Document Identifier: CMS-DEV-V1.2.4
Target Audience: Software Engineers, Full-Stack Developers, DevOps Engineers
Date: September 2026
Status: Approved for Handover

---

1. Introduction & Architecture Overview

The **Pakistan Navy Complaint Management System (CMS)** is built on the **Laravel Framework** (PHP 8.2) following modern MVC and service-oriented patterns. It exposes both server-rendered Blade administrative views and high-throughput RESTful JSON endpoints for mobile and field clients.

1.1 Key Engineering Tenets

- **Clean Code & Strict Typing:** Strict adherence to PSR-12 coding style guidelines and comprehensive DocBlock documentation.
- **Granular Decoupling:** Complete separation of concerns between Controllers, Eloquent Models, and Data Transfer layers.
- **Deterministic Deployments:** Zero-downtime, atomic symlink/folder swap deployments on Linux cPanel VPS environments.

---

2. Local Development Environment Setup

2.1 System Prerequisites

- **PHP:** Version 8.2 or higher with extensions: `pdo_mysql`, `mbstring`, `openssl`, `bcmath`, `curl`, `fileinfo`, `gd`.
- **Composer:** Version 2.5 or higher.
- **Node.js & NPM:** Node.js v18.x or v20.x LTS, NPM v9+.
- **Database:** MySQL 8.0 or MariaDB 10.6+ (InnoDB storage engine).
- **Local Web Server:** XAMPP, WampServer, Laravel Herd, or native PHP CLI server.

2.2 Step-by-Step Installation

```
# Step 1: Clone the repository
git clone https://github.com/nexertech/CMS.git c:\xampp\htdocs\CMS
cd c:\xampp\htdocs\CMS

# Step 2: Install PHP dependencies
composer install

# Step 3: Install Node dependencies & build front-end assets
npm install
npm run build

# Step 4: Setup Environment Configuration
cp .env.example .env

# Generate unique application cryptographic key
php artisan key:generate

# Step 5: Configure Database Credentials in .env
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=cms_db
# DB_USERNAME=root
# DB_PASSWORD=

# Step 6: Execute Database Migrations and Core Seeders
php artisan migrate --seed

# Step 7: Create Public Storage Symlink (for complaint photos and documents)
php artisan storage:link

# Step 8: Launch Local Development Server
php artisan serve
# Application is now accessible at http://127.0.0.1:8000
```

---

3. Directory Structure Walkthrough

```
CMS/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Admin/                 # Administrative web controllers (Users, Roles, Houses, etc.)
│   │   │   ├── Api/                   # RESTful mobile API controllers (ComplaintApiController, etc.)
│   │   │   ├── Auth/                  # Multi-guard authentication controllers
│   │   │   ├── ComplaintController.php # Core complaint engine & status coordinator
│   │   │   ├── SpareController.php     # Inventory & stock requisition controller
│   │   │   └── ThemeController.php     # Tri-mode accessibility theme controller
│   │   └── Middleware/
│   │       ├── CheckRolePermission.php # Granular exact-match RBAC enforcement
│   │       ├── ManualAuthMiddleware.php# Mobile token validator
│   │       └── DeviceCheckMiddleware.php# Registered device IMEI/UUID validator
│   └── Models/                        # 31 Eloquent models with relations & soft deletes
├── config/                            # Framework configuration (auth, app, database, cors)
├── database/
│   ├── migrations/                    # 34 chronological database migration files
│   └── seeders/                       # Seeders for default Superadmin, Roles & Categories
├── public/
│   ├── build/                         # Compiled Vite production JS/CSS bundles
│   └── index.php                      # Application entry point
├── resources/
│   ├── css/                           # Custom styling, themes.css (Light/Dark/Night variables)
│   ├── js/                            # Theme manager and frontend interactivity
│   └── views/                         # Blade template hierarchy (layouts, admin, components)
├── routes/
│   ├── api.php                        # Throttled mobile REST endpoints
│   ├── auth.php                       # Authentication routes
│   ├── frontend.php                  # Public / resident portal routes
│   └── web.php                        # Main administrative routes guarded by RBAC
└── storage/                           # Application logs, file attachments, and cache
```

---

4. Core Architecture & Subsystems

4.1 Complaint State Machine (`app/Models/Complaint.php`)

Status values are persisted as `TINYINT` values for query efficiency and memory conservation:

```
class Complaint extends Model
{
    const STATUS_IN_PROGRESS = 0;
    const STATUS_RESOLVED = 1;
    const STATUS_UNASSIGNED = 2; // Default for new complaints
    const STATUS_ASSIGNED = 3;
    const STATUS_WORK_PERFORMA = 4;
    const STATUS_MAINT_PERFORMA = 5;
    const STATUS_WORK_PRICED_PERFORMA = 6;
    const STATUS_MAINT_PRICED_PERFORMA = 7;
    const STATUS_PRODUCT_NA = 8;
    const STATUS_UN_AUTHORIZED = 9;
    const STATUS_BARRACK_DAMAGES = 10;
    const STATUS_DOOR_LOCK = 11;
}
```

4.2 Material Performa & Stock Deduction Subsystem

When a technician requires spares:
1. `SpareApprovalPerforma` is created with type (`work_performa` , `maint_performa` , or `priced`).
2. Multiple `SpareApprovalItem` records attach item IDs and requested quantities.

3. Upon approval (`status = 'approved'`), database transaction:
* Deducts quantity from `spares.current_stock` .
* Creates an entry in `complaint_spares` linking the item to the complaint.
* Logs transaction to `spare_stock_logs` for audit.

4.3 Tri-Mode Theme Engine

- The application CSS declares CSS Custom Properties (Variables) inside `resources/css/themes.css` :
- `.theme-light` : Standard crisp daylight UI.
- `.theme-dark` : High contrast dark mode.
- `.theme-night` : Low blue-light mode (amber/sepia undertones) optimized for 24/7 control room desks.
- `.theme-auto` : Evaluates browser `prefers-color-scheme` .
- Handled client-side by `resources/js/theme-manager.js` and server-side via `ThemeController::getThemeClass(request())` .

---

5. Git Workflow & Release Protocols

5.1 Branching Strategy

- **`Amjad_branch` (Feature Branch):** All feature development, bug fixes, and testing occur here.
- **`deploy` (Production Branch):** Clean, stable branch containing only verified production code, compiled assets (`public/build`), and optimized autoloaders.
- **Tags (`vX.Y.Z`):** Immutable, annotated git release tags deployed to the VPS.

5.2 Local Release Tagging Protocol

Prior to creating a release:

```
# 1. Compile production JS/CSS assets
npm run build

# 2. Optimize PHP autoloader
composer install --no-dev --optimize-autoloader

# 3. Switch to deploy branch and merge changes
git checkout deploy
git pull origin deploy
git merge Amjad_branch

# 4. Force add compiled build artifacts
git add -f public/build vendor
git commit -m "Build: Release v1.2.4 compiled production assets"

# 5. Tag and push
git tag -a v1.2.4 -m "Release v1.2.4: Granular RBAC and Route Guarding"
git push origin deploy --tags
```

---

6. Official Production Deployment Runbook (VPS `paknavy`)

> **Server Environment:** JazzCloud VPS (cPanel Linux)
> **Execution User:** `paknavy` *(Never execute commands as root or with sudo)*
> **Production Live Path:** `/home/paknavy/public_html` *(NEVER `/home/paknavy/CMS`)*
> **PHP Binary:** `/opt/cpanel/ea-php82/root/usr/bin/php`
> **Deployment Architecture:** Staging Clone + Atomic Folder Swap

Step-by-Step Server Swap Execution Script:

```
# 1. Navigate to home and clear old staging/rollback folders
cd /home/paknavy
rm -rf public_html_new public_html_old

# 2. Create dated safety backup of current live deployment
cp -r public_html public_html_backup_$(date +%F)

# 3. Clone tagged release into staging directory
git clone --branch v1.2.4 https://github.com/nexertech/CMS.git public_html_new
cd /home/paknavy/public_html_new

# 4. Copy live environment configuration & persistent storage
cp /home/paknavy/public_html/.env .env
rm -rf storage
cp -r /home/paknavy/public_html/storage ./

# 5. Restore Master Security .htaccess (Prevents raw IP browsing and locks server files)
cp /home/paknavy/deploy-assets/app-root-htaccess.txt /home/paknavy/public_html_new/.htaccess

# 6. Set correct permissions on writable folders
chmod -R 775 storage bootstrap/cache
chown -R paknavy:paknavy storage bootstrap/cache

# 7. Run database migrations (ONLY if release contains DB changes)
# /opt/cpanel/ea-php82/root/usr/bin/php artisan migrate --force

# 8. Atomic Folder Swap (Instantaneous transition without downtime)
cd /home/paknavy
mv public_html public_html_old
mv public_html_new public_html

# 9. Clear & Rebuild Application Caches (Executed inside live public_html)
cd /home/paknavy/public_html
/opt/cpanel/ea-php82/root/usr/bin/php artisan config:clear
/opt/cpanel/ea-php82/root/usr/bin/php artisan route:clear
/opt/cpanel/ea-php82/root/usr/bin/php artisan view:clear
/opt/cpanel/ea-php82/root/usr/bin/php artisan cache:clear

/opt/cpanel/ea-php82/root/usr/bin/php artisan config:cache
/opt/cpanel/ea-php82/root/usr/bin/php artisan route:cache
/opt/cpanel/ea-php82/root/usr/bin/php artisan view:cache

# 10. Record Current Live Version
echo "v1.2.4" > /home/paknavy/current-live-version.txt
```

Emergency Rollback Sequence (Instant Reversal)

If unexpected issues occur immediately post-deploy:

```
cd /home/paknavy
mv public_html public_html_failed
mv public_html_old public_html
cd /home/paknavy/public_html
/opt/cpanel/ea-php82/root/usr/bin/php artisan config:clear
/opt/cpanel/ea-php82/root/usr/bin/php artisan route:clear
```

© 2026 Pakistan Navy CMS • Nexertech Engineering